Web Technology

Full Marks : 70 **SEM-III/CSE 2022 (Odd)**

Group - A

Q.1. Choose the most suitable answer from the following options **(1 x 20)**

(i) Are the negative values allowed in padding property?
- (a) Yes
- **(b) No**
- (c) Can't say
- (d) May be

Ans: b

(ii) The CSS properly used to specify the transparency of an element is …….
- **(a) Opacity**
- (b) Filter
- (c) Visibility
- (d) Overlay

Ans: a

(iii) The CSS property used to draw a line around the element outside the border?
- (a) Border
- **(b) Outline**
- (c) Padding
- (d) Line

Ans: b

(iv) What does XML stand for?
- (a) Extra modern link
- **(b) Extensive markup language**
- (c) Example markup language
- (d) X-markup language

Ans: b

(v) Which statement is true?
- **(a) All xml elements must have a closing tag**
- (b) All xml documents must have a DTD
- (c) All xml elements must be lower case
- (d) All the statement are true

Ans: a

(vi) In xml document comments are given as?
- (a) <!-- -- !>
- **(b) <!---->**
- (c) </-..>
- (d) <?...?

Ans: b

(vii) In which part of the HTML metadata is contained?
- **(a) Head tag**
- (b) Title tag
- (c) HTML tag
- (d) Body tag

Ans: a

(viii) Which element is used to get highlighted text in HTML5
- (a) <u>
- **(b) <mark>**
- (c) < highlight
- (d) <b>

Ans: b

(ix) Which of the following is used to read an HTML page and send it?

(a) Web server
(b) Web network
(c) Web browser
(d) Web matrix

Ans: c

(x) Which of the following HTML tag is used to add a row in a table?

(a) <th>
(b) <td>
(c) <tr>
(a) (d)<tt>

Ans: c

(xi) Which tag is used to create a dropdown in HTML form?

(a) <input>
(b) <select>
(c) <text>
(d) <textarea>

Ans: b

(xii) Which of the following extension is used to save on HTML file?

(a) hi
(b) .h
(c) .htl
(d) .html

Ans: d

(xiii) Which of the following is not JavaScript data types?

(a) Null type
(b) Undefined type
(c) Number type
(d) All of the mentioned

Ans: None of the above. (Correct option is not available) ***Note:*** *All of the mentioned data types (null, undefined, and number) are valid JavaScript data types.*

(xiv) Which of the following scoping type does JavaScript use?

(a) Sequential
(b) Segmental
(c) Lexical
(d) Literal

Ans: c

(x) What will be the output of the following JavaScript code?

```
int a = 0;
for (a;a<5; a++);
console.log (a);
```

(a) 4
(b) 5
(c) 0
(d) Error

Ans: b

(xvi) Which of the following is the property that is triggered in response to IS errors?

(a) Onclick
(b) Onerror
(c) Onmessage
(e) Onexception

Ans: b

(xvii) Server are computers that provide resources to other computers connected to a:

(a) Client

(b) Mainframe
(c) Supercomputer
(d) Network

Ans: d

(xviii) A program that is used to view websites is called a:

(a) Browser
(b) Web viewer
(c) Spread sheet
(d) Word processor

Ans: a

(xix) Which of the following is not a type of broadband internet connection.

(a) Satellite
(b) DSL
(c) Dial up
(d) Cable

Ans: c

(xx) A computer on internet is identified by:

(a) Email address
(b) IP address
(c) Street address
(d) Server address

Ans: b

Group - B

Q. 2. Define XML schema. (4)

Ans: XML Schema is commonly known as XML Schema Definition (XSD). It is used to describe and validate the structure and the content of XML data. XML schema defines the elements, attributes and data types. Schema element supports Namespaces. It is similar to a database schema that describes the data in a database. It is like DTD but provides more control on XML structure. An XML Schema describes the structure of an XML document, just like a DTD. An XML document with correct syntax is called "Well Formed". An XML document validated against an XML Schema is both "Well Formed" and "Valid". There are so many schema languages which are used now a days for example Relax- NG and XSD (XML schema definition).

There are two types of data types in XML schema.

- **simpleType** - It allows to have text-based elements. It contains less attributes, child elements, and cannot be left empty.
- **complexType** - It allows to hold multiple attributes and elements. It can contain additional sub elements and can be left empty.

XML Schema Coding : student.xsd

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.sbte.edu/2022/XMLSchema"
targetNamespace="http://www. sbte.edu"
xmlns="http://www.sbte.edu"
elementFormDefault="qualified">
<xs:element name="student">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="firstname" type="xs:string"/>
      <xs:element name="lastname" type="xs:string"/>
      <xs:element name="email" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>
```

Where,

<xs:element name="student"> : It defines the element name employee.

<xs:complexType> : It defines that the element 'student' is complex type.
<xs:sequence> : It defines that the complex type is a sequence of elements.
<xs:element name="firstname" type="xs:string"/> : It defines that the element 'firstname' is of string/text type.
<xs:element name="lastname" type="xs:string"/> : It defines that the element 'lastname' is of string/text type.
<xs:element name="email" type="xs:string"/> : It defines that the element 'email' is of string/text type.

OR

Q. 2. Define ordered list in HTML with an example. (4)

Ans: HTML Ordered List or Numbered List displays elements in numbered format. The HTML ol tag is used for ordered list. We can use ordered list to represent items either in numerical order format or alphabetical order format, or any format where an order is emphasized. There can be different types of numbered list:

- Numeric Number (1, 2, 3)
- Capital Roman Number (I II III)
- Small Romal Number (i ii iii)
- Capital Alphabet (A B C)
- Small Alphabet (a b c)

To represent different ordered lists, there are 5 types of attributes in **<ol>** tag.

Type	Description
Type "1"	This is the default type. In this type, the list items are numbered with numbers.
Type "I"	In this type, the list items are numbered with upper case roman numbers.
Type "i"	In this type, the list items are numbered with lower case roman numbers.
Type "A"	In this type, the list items are numbered with upper case letters.
Type "a"	In this type, the list items are numbered with lower case letters.

HTML Ordered List Example

```
<ol>
    <li>HTML</li>
    <li>Java</li>
    <li>JavaScript</li>
    <li>SQL</li>
</ol>
```

Output:
HTML
Java
JavaScript
SQL

Q. 3. Define DTD in XML. (4)

Ans: A **Document Type Definition** (**DTD**) describes the tree structure of a document and something about its data. It is a set of markup affirmations that actually define a type of document for the SGML family, like GML, SGML, HTML, XML. A DTD can be declared inside an XML document as inline or as an external recommendation. DTD determines how many times a node should appear, and how their child nodes are ordered.
There are 2 data types, PCDATA and CDATA

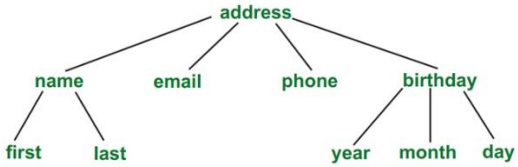
- PCDATA is parsed character data.
- CDATA is character data, not usually parsed.

Syntax:

```
<!DOCTYPE element DTD identifier
[
    first declaration
    second declaration
    :
```

:
nth declaration
] >

Example:



DTD for the above tree is:

XML document with an internal DTD:

```
<?xml version="1.0"?>
  <!DOCTYPE address [
      <!ELEMENT address (name, email, phone, birthday)>
      <!ELEMENT name (first, last)>
      <!ELEMENT first (#PCDATA)>
      <!ELEMENT last (#PCDATA)>
      <!ELEMENT email (#PCDATA)>
      <!ELEMENT phone (#PCDATA)>
      <!ELEMENT birthday (year, month, day)>
      <!ELEMENT year (#PCDATA)>
      <!ELEMENT month (#PCDATA)>
      <!ELEMENT day (#PCDATA)>
    ]>
    <address>
        <name>
            <first>Ajay</first>
            <last>Sharma</last>
        </name>
        <email>sharmaajay@gmail.com</email>
        <phone>9661116218</phone>
        <birthday>
            <year>1987</year>
            <month>June</month>
            <day>23</day>
        </birthday>
    </address>
```

XML document with an external DTD:

```
<?xml version="1.0"?>
<!DOCTYPE address SYSTEM "address.dtd">
    <address>
        <name>
            <first>Ajay</first>
            <last>Sharma</last>
        </name>
        <email>sharmaajay@gmail.com</email>
        <phone>9661116218</phone>
        <birthday>
            <year>1987</year>
            <month>June</month>
            <day>23</day>
        </birthday>
    </address>
```

Output:

```
- <address>
  - <name>
      <first>Ajay </first>
      <last>Sharma</last>
    </name>
    <email>sharmaajay@gmail.com</email>
    <phone>9661116218 </phone>
  - <birthday>
      <year>1987</year>
      <month>June</month>
      <day>23</day>
    </birthday>
</address>
```

OR

Q. 3. Explain about cascading style sheets with an example. (4)

Ans: **CSS** stands for **C**ascading **S**tyle **S**heets. **Cascading Style Sheets** (**CSS**) is a stylesheet language used to describe the presentation of a document written in HTML or XML CSS describes how elements should be rendered on screen, on paper, in speech, or on other media and variations in display for different devices and screen sizes. It can control the layout of multiple web pages all at once. External style sheets are stored in **".css"** files. CSS is used to define styles for your web pages, including the design, layout

CSS Syntax

A CSS comprises of style rules that are interpreted by the browser and then applied to the corresponding elements in your document. A style rule is made of three parts:

- **Selector:** A selector is an HTML tag at which a style will be applied. This could be any tag like <h1> or <table> etc.
- **Property:** A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be color, border, etc.
- **Value:** Values are assigned to properties. For example, color property can have the value either red or #F1F1F1 etc.

CSS Style Rule Syntax is as follows: **selector { property: value }**

For Example

```
<style type= "text/css">
  p
  {
    text-align: justify;
    background-color: red;
  }
</style>
```

Example of External CSS:

```
<html>
  <head>  <link rel = "stylesheet" type = "text/css" href = "external.css">  </head>
  <body>
    <p>This is Internal CSS.</p>
  </body>
</html>
external.css:
p
  {
    text-align: justify;
    background-color: red;
  }
```

Q. 4. What is JavaScript? What are the features of JavaScript? (4)

Ans: JavaScript - JavaScript was created in 1995 by Brendan Eich. **JavaScript** is a lightweight text-based scripting language that can use for server-side and client-side in .Net applications. JavaScript program code enables to create dynamically displaying timely content updates, interactive maps, animated 2D/3D graphics, scrolling video jukeboxes, images, etc. and pretty much everything else. It is the third layer of the layer cake of standard web technologies, two of which (HTML and CSS). A very common use of JavaScript is to dynamically modify HTML and CSS to update a user interface, via the Document Object Model (DOM) API. Errors may occur if JavaScript is loaded and run before the HTML and CSS that it is intended to modify.

Example 1: JavaScript Program to Print Hello World

```
<html>
    <body>
        <script type="text/javascript">
            alert("Hello World");
        </script>
    </body>
</html>
```

Output

Hello World

Features of JavaScript

According to a recent survey conducted by Stack Overflow, JavaScript is the most popular language on earth. With advances in browser technology and JavaScript having moved into the server with Node.js and other frameworks, JavaScript is capable of so much more. Here are a few things that we can do with JavaScript:

- JavaScript was created in the first place for DOM manipulation. Earlier websites were mostly static, after JS was created dynamic Web sites were made.
- Functions in JS are objects. They may have properties and methods just like another object. They can be passed as arguments in other functions.
- Can handle date and time.
- Performs Form Validation although the forms are created using HTML.
- No compiler is needed.

OR

Q. 4. Define ANCHOR TAG with an example. (4)

Ans: The HTML anchor tag defines a hyperlink that links one page to another page. It can create hyperlink to other web page as well as files, location, or any URL. The "href" attribute is the most important attribute of the HTML a tag. and which links to destination page or URL.

href attribute of HTML anchor tag

The **href** attribute is used to define the address of the file to be linked. In other words, it points out the destination page. The syntax of HTML anchor tag is given below.

** Link Text **

Example of HTML anchor tag.

Click for Second Page

Specify a location for Link using target attribute

To open that link to another page then use target attribute of **<a>** tag. With the help of this link will be open in next page.

```
<!DOCTYPE html>
<html>
  <head>
     <title></title>
  </head>
  <body>
     <p>Click on <a href="https://sbte.bih.nic.in/" target="_blank">
         this-link </a>to go on home page of STATE BOARD OF TECHNICAL EDUCATION.</p>
</body>
</html>
```

Appearance of HTML anchor tag

- An **unvisited link** is displayed underlined and blue.
- A **visited link** displayed underlined and purple.
- An **active link** is underlined and red.

Q. 5. Define table tag and their attributes with an example. (4)

Ans: A **Table** is created using three basic tags: **table**, **tr**, and **td**. The tag **table** is the container of the whole table. The **opening tag <table>** tells the browser that a table is to be created. Table specific properties are also defined here. The **closing tag </table>** tells the browser that it is the end of the table. Everything else is contained within the **<table>** and **</table>** **tag pair**. So, a table always looks like this:

```
<table>
  ……
</table>
```

HTML table tags

Tag	Meaning
table	Represents the whole table
tr	Represents a row
td	Represents a cell in a row
th	Column header
caption	Title of the table

A table consists of a number of rows each of which is made up of a number of cells. The tag **<tr>** creates a **row**. The total number of rows in a table is equal to the number of **<tr>** elements. The opening tag **<tr>** and closing tag **</tr>** indicate the beginning and ending of a table row, respectively.

```
<table>
    <tr>….</tr>
    <tr>….</tr>
      …….
    <tr>….</tr>
</table>
```

A table or a row cannot contain data. To add data, cells are created within a row. Each cell in a row is defined by the tag **<td>**. The number of columns in a row is equal to the number of **<td>** elements. The content of a cell is written within the **<td>** and **</td>** tag pair. Textual as well as graphical content can be used in a cell. Usual tags such as **<p>**, **
, **<i>, **<b>**, and **<u>** may be used within a cell.

The following code creates one such table:

```
<table>
    <tr> <td>82</td> <td>80</td> </tr>
    <tr> <td>80</td> <td>75</td> </tr>
</table>
```

OR

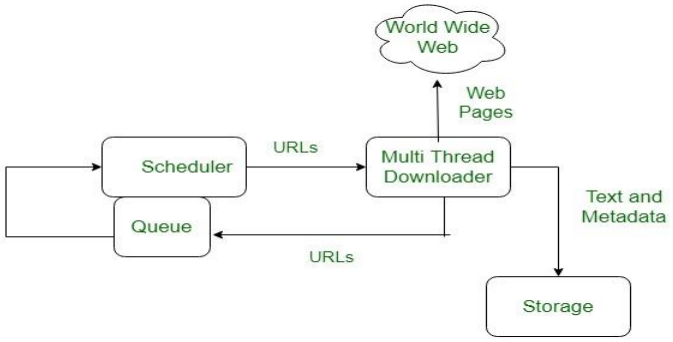
Q. 5. Explain Architecture of WWW in detail. (4)

Ans: The World Wide Web is abbreviated as WWW and is commonly known as the web. WWW can be defined as the collection of different websites around the world, containing different information shared via local servers (or computers). The Internet is a worldwide connectivity of hundreds of thousands of computers of various types that belong to multiple networks.

System Architecture:

From the user's point of view, the web consists of a vast, worldwide connection of documents or web pages. Each page may contain links to other pages anywhere in the world. The pages can be retrieved and viewed by using browsers of which internet explorer, Netscape Navigator, Google Chrome, etc are the popular ones. The browser

fetches the page requested interprets the text and formatting commands on it, and displays the page, properly formatted, on the screen.
The basic model of how the web works is shown in the figure below. Here the browser is displaying a web page on the client machine. When the user clicks on a line of text that is linked to a page on the abd.com server, the browser follows the hyperlink by sending a message to the abd.com server asking it for the page.



Here the browser displays a web page on the client machine when the user clicks on a line of text that is linked to a page on abd.com, the browser follows the hyperlink by sending a message to the abd.com server asking for the page.

Q. 6. What is the scope of variables in JavaScript. (4)

Ans: The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes.

- **Global Variables** − A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- **Local Variables** − A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable. Take a look into the following example.

```
<html>
   <body onload = checkscope();>
      <script type = "text/javascript">
         <!--
            var myVar = "global";     // Declare a global variable
            function checkscope( ) {
               var myVar = "local";    // Declare a local variable
               document.write(myVar);
            }
         //-->
      </script>
   </body>
</html>
```

OR

Q. 6. Explain HTML document to provide a form that collect name and telephone numbers. (4)

Ans:

```
<!DOCTYPE html>
<html>
<head>
    <title>
        Telephone number Entry Form
    </title>
    <style>
        #Poly_p {
        font-size: 30px;
            color: green;
```

```
            }
            h1,
            h2 {
                font-family: impact;
            }
        </style>
</head>
<body style="text-align: center">
        <h1 style="color: green">Government Polytechnic, Bihar</h1>
        <h2>
            To add a Telephone number into a Form using HTML5
        </h2>
        <form>
            Name:
                <input type="text"
                    placeholder="Enter your name here--" />
                <br />
            Address:
                <input type="text"
                    placeholder="Enter your permanent Address" />
                <br />
            Phone No.:
                <input type="tel"
                    placeholder="Enter phone number" />
                <br />
                <br />
                <button>Submit</button>
        </form>
</body>
</html>
```

Group - C

Q. 7. State the differences between XML schema and DTD with suitable examples. (6)

Ans:

Document Type Definition (DTD)

DTD stands for Document Type Definition and it is a document which defines the structure of an XML document. It is used to describe the attributes of XML language precisely. It can be classified into two types namely internal DTD and external DTD. It can be specified inside a document or outside a document. DTD mainly checks the grammar and validity of a XML document. It checks that a XML document has a valid structure or not.

Program Name: book.xml

```
<?xml version="1.0"?>
<!DOCTYPE Library SYSTEM "book.dtd">
<library>
<bookname>Introduction to XML</bookname>
<authorname>Bala Gurusamy</authorname>
<email>educba@hotmail.com</email>
</library>
```

Coverted DTD Code Name: book.dtd

```
<!DOCTYPE Library
[
<!ELEMENT library (bookname, authorname,email)>
<!ELEMENT bookname (#PCDATA)>
<!ELEMENT authorname (#PCDATA)>
<!ELEMENT email (#PCDATA)>
]>
```

XML Schema Definition (XSD)

XSD stands for XML Schema Definition and it is a way to describe the structure of a XML document. It defines the rules for all the attributes and elements in a XML document. It can also be used to generate the XML documents. It also checks the vocabulary of the document. It doesn't require processing by a parser. XSD checks for the correctness of the structure of the XML file. XSD was first published in 2001 and after that it was published in 2004.

Program Name: book.xml

```
<?xml version="1.0"?>
<!DOCTYPE Library SYSTEM "book.dtd">
<library>
<bookname>Introduction to XML</bookname>
<authorname>Bala Gurusamy</authorname>
<email>educba@hotmail.com</email>
</library>
```

Program Code: book.xsd

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="library">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="bookname" type="xs:string" />
        <xs:element name="authorname" type="xs:string" />
        <xs:element name="email" type="xs:string" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Difference between Document Type Definition (DTD) and XML Schema Definition (XSD)

S.NO.	DTD	XSD
1.	DTD are the declarations that define a document type for SGML.	XSD describes the elements in a XML document.
2.	It doesn't support namespace.	It supports namespace.
3.	It is comparatively harder than XSD.	It is relatively more simpler than DTD.
4.	It doesn't support datatypes.	It supports datatypes.
5.	SGML syntax is used for DTD.	XML is used for writing XSD.
6.	It is not extensible in nature.	It is extensible in nature.
7.	It doesn't give us much control on structure of XML document.	It gives us more control on structure of XML document.
8.	It specifies only the root element.	Any element which is made global can be done as root as markup validation.
9.	It doesn't have any restrictions on data used.	It specifies certain data restrictions.
10.	It is not much easy to learn .	It is simple in learning.
11.	File here is saved as .dtd	File in XSD is saved as .xsd file.
12.	It is not a strongly typed mechanism.	It is a strongly typed mechanism.
13.	It uses #PCDATA which is a string data type.	It uses fundamental and primitive data types.

OR

Q. 7. Write the differences between CSS and XSL. (6)

Ans:

XSL	CSS
XSL uses the XML notation that is helpful in writing the codes and the tools are provided in greater extent	CSS doesn't use any of the XML notations but has its own.
XSL is having the formatting object tree setup differently from the source tree	CSS is having the source tree and the formatting object tree the same.
XSL can't provide the inheritance of the source tree using the formatting properties	CSS provides the inheritance of the formatting object that is related to the source tree.
XSL is not supported by many of the web browsers	CSS can be read by modern web browsers.

Q. 8. What is webserver? Explain the feature of Web server. (6)

Ans: Webserver : A web server is a dedicated computer responsible for running websites sitting out on those computers somewhere on the Internet. They are specialized programs that circulate web pages as summoned by the user. The primary objective of any web server is to collect, process and provide web pages to the users.

This intercommunication of a web server with a web browser is done with the help of a protocol named HTTP (Hypertext Transfer Protocol). These stored web pages mostly use static content, containing HTML documents, images, style sheets, text files, etc. However, web servers can serve static as well as dynamic contents. Web Servers also assists in emailing services and storing files. Therefore, it also uses SMTP (Simple Mail Transfer Protocol) and FTP (File Transfer Protocol) protocols to support the respective services. Web servers are mainly used in web hosting or hosting the website's data and running web-based applications.

Features of Webserver:

1. Web Server can support enlarge data storage support, so it is capable to make multiple websites.
2. Easy to configure log file set up, enabling where to hold all log files. (Log files help to analyses web traffic and more)
3. It helps to control bandwidth to regulate network traffic, so due to this it can avoid the down time while flowing high volume web traffic.
4. Easy to make FTP websites, because it helps to move enlarge files from one site to other sites.
5. Easy to set up website configuration and directory security
6. Easy to make virtual directories, and then help to map them along with physical directories.
7. Easy to set up of custom error pages configuration that means it helps to view user friendly error messages on your website, when your website is getting any issues like as 404 Error will be displayed if web pages do not present.
8. Can be specified default documents for example if any file has not specified with its name, then default documents will be displayed.
9. It is enabled with Server-side web scripting so it allows user to make dynamic websites. Few types of server-side scripting languages are PHP, ASP, Ruby, Perl, Python, and more.

OR

Q. 8. How we design a good website? List the criterion to define a better web. (6)

Ans: A good website design should fulfil its intended function by conveying its particular message whilst simultaneously engaging the visitor. Several factors such as consistency, colours, typography, imagery, simplicity, and functionality contribute to good website design. Creating a great user experience involves making sure the website design is optimised for usability and how easy is it to use.

Some of the factors which define criterion for good website design are:

1. WEBSITE PURPOSE: A website needs to accommodate the needs of the user. Having a simple clear intention on all pages will help the user interact with what have to offer. There are many different purposes that websites may have but there are core purposes common to all websites:

- Describing Expertise
- Building Your Reputation

- Generating Leads
- Sales and After Care

2. SIMPLICITY: Simplicity is the best way to go when considering the user experience and the usability of your website. Some of the ways are to achieve simplicity through design. Colour has the power to communicate messages and evoke emotional responses. Typography has an important role to play on your website. It commands attention and works as the visual interpretation of the brand voice. Imagery is every visual aspect used within communications. This includes still photography, illustration, video and all forms of graphics.

3. NAVIGATION: Navigation is the way finding system used on websites where visitors interact and find what they are looking for. Website navigation is key to retaining visitors. If the website navigation is confusing visitors will give up and find what they need elsewhere. Keeping navigation simple, intuitive and consistent on every page is key.

4. F-SHAPED PATTERN READING: The F-based pattern is the most common way visitors scan text on a website. Eye-tracking studies have found that most of what people see is in the top and left areas of the screen. The F-shaped layout mimics our natural pattern of reading in the West (left to right and top to bottom). An effectively designed website will work with a reader's natural pattern of scanning the page.

5. VISUAL HIERARCHY: Visual hierarchy is the arrangement of elements in order of importance. This is done either by size, colour, imagery, contrast, typography, whitespace, texture and style. One of the most important functions of visual hierarchy is to establish a focal point; this shows visitors where the most important information is.

6. CONTENT: An effective website has both great design and great content. Using compelling language great content can attract and influence visitors by converting them into customers.

7. GRID BASED LAYOUT: Grids help to structure your design and keep your content organised. The grid helps to align elements on the page and keep it clean. The grid-based layout arranges content into a clean rigid grid structure with columns, sections that line up and feel balanced and impose order and results in an aesthetically pleasing website.

8. LOAD TIME: Waiting for a website to load will lose visitors. Nearly half of web visitors expect a site to load in 2 seconds or less and they will potentially leave a site that isn't loaded within 3 seconds. Optimising image sizes will help load your site faster.

9. MOBILE FRIENDLY: More people are using their phones or other devices to browse the web. It is important to consider building your website with a responsive layout where your website can adjust to different screens.

Q. 9. What is Angular JS? What are the main advantages and disadvantages of Angular JS? (6)

Ans: **AngularJS:** **AngularJS** is an open-source JavaScript framework of Google to make **SPA** (**single-page applications or one-page web applications or dynamic web app**) and which was recently made available to the public under the MIT license. AngularJS is a JavaScript framework that extends HTML DOM to make it dynamic, and develops its own tags and attributes. AngularJS is the client-side technology, entirely written in JavaScript that only require HTML, CSS, and JavaScript. It is an extensible framework that wants and pushes towards a structured development.

It is based on MVW framework (Model-View-Whatever), where whatever means Whatever Works for You and allows you to build well structured, easily testable, and maintainable front-end applications. The AngularJS can be used both as MVC and MVVM framework.

AngularJS was originally developed in 2009 by Misko Hevery and Adam Abrons at BratTech LLC(California). Its latest version is 1.4.3.

AngularJS is built around the idea that declarative programming should be used for creating user interfaces and cabling software components, while the imperative programming is excellent for expressing business logic. The framework adapts and extends the traditional HTML to better manage dynamic content to through the two-way data-binding (data link in both directions), which allows automatic synchronization of models and views.

AngularJS Advantages

Here are the advantages of AngularJS:

- Since it's an open-source framework, user can expect the number of errors or issues to be minimal.
- Two-way binding – Angular.js keeps the data and presentation layer in sync. Now don't need to write additional JavaScript code to keep the data in HTML code and data later in sync. Angular.js will automatically do this. It is just need to specify which control is bound to which part of user model.

- Routing – Angular can take care of routing which means moving from one view to another. This is the key fundamental of single page applications; wherein user can move to different functionalities in web application based on user interaction but still stay on the same page.
- Angular supports testing, both Unit Testing, and Integration Testing.
- It extends HTML by providing its own elements called directives. At a high level, directives are markers on a DOM element (such as an attribute, element name, and comment or CSS class) that tell AngularJS's HTML compiler to attach a specified behavior to that DOM element. These directives help in extending the functionality of existing HTML elements to give more power to web application.

Disadvantages of AngularJS

- AngularJS is vast and complicated. Although it offers several ways to accomplish the same task, it doesn't clearly say which way is the best for a certain task.
- Different coding styles of different developers may complicate things so far integration of different components into an entire solution is concerned.
- The entire lifecycle of an Angular application is intricate, mastering on which one would require reading code.
- Link and compile are not spontaneous. Specific cases, such as a - collision between directives and recursion in compile etc. may create confusion.
- Since Angular implementation offers a poor scalability, it can be problematic with the time and as the project grows. With the growing phase of development, you need to through old implementations and to create new versions by adhering different ways.
- Since Java Script is the only framework, an application developed in AngularJS are not secure. To secure the application, server-side authentication and authorization become inevitable.
- In case Java Script is disabled, the user will only be able to see the basic page.

OR

Q.9. What are JavaScript data types? Explain with example. (6)

Ans: JavaScript Datatypes: One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language. JavaScript provides different **data types** to hold different types of values. There are two types of data types in JavaScript.

1. Primitive data type
2. Non-primitive (reference) data type

JavaScript is a **dynamic type language**; means there is no need to specify type of the variable because it is dynamically used by JavaScript engine.

1. JavaScript primitive data types

There are 7 types of primitive data types in JavaScript. They are as follows:

Data Types	Description	Example
String	represents textual data	'hello', "hello world!" etc
Number	an integer or a floating-point number	3, 3.234, 3e-2 etc.
BigInt	an integer with arbitrary precision	900719925124740999n , 1n etc.
Boolean	Any of two values: true or false	true and false
undefined	a data type whose variable is not initialized	let a;
null	denotes a null value	let a = null;
Symbol	data type whose instances are unique and immutable	let value = Symbol('hello');
Object	key-value pairs of collection of data	let student = { };

2. JavaScript non-primitive data types

The non-primitive data types are as follows:

Data Type	Description	Example

Object	key-value pairs of collection of data ; represents instance through which we can access members	let student = { };
Array	represents group of similar values	
RegExp	represents regular expression	

JavaScript String

String is used to store text. In JavaScript, strings are surrounded by quotes:

Single quotes: 'Hello'
Double quotes: "Hello"
Backticks: `Hello`

For example,

```
const name = 'ram';
const name1 = "hari";
const result = `The names are ${name} and ${name1}`;
```

Single quotes and double quotes are practically the same. Backticks are generally used when need to include variables or expressions into a string. This is done by wrapping variables or expressions with ${variable or expression}

JavaScript Number

Number represents integer and floating numbers (decimals and exponentials).

For example,

```
const number1 = 3;
const number2 = 3.433;
const number3 = 3e5      // 3 * 10^5
```

A number type can also be +Infinity, -Infinity, and NaN (not a number).

JavaScript BigInt

In JavaScript, Number type can only represent numbers less than (253 - 1) and more than -(253 - 1). However, to use a larger number than that, the BigInt data type is used.

A BigInt number is created by appending **n** to the end of an integer

```
const value1 = 900719925124740998n;   // BigInt value
```

JavaScript Boolean

This data type represents logical entities. Boolean represents one of two values: true or false. It is easier to think of it as a yes/no switch.

For example,

```
const dataChecked = true;
const valueCounted = false;
```

JavaScript undefined

The undefined data type represents value that is not assigned. If a variable is declared but the value is not assigned, then the value of that variable will be undefined.

For example,

```
let name;
console.log(name); // undefined
let name = undefined;
console.log(name); // undefined
```

JavaScript null

In JavaScript, null is a special value that represents empty or unknown value.

For example,

```
const number = null;
```

JavaScript Symbol

This data type was introduced in a newer version of JavaScript (from ES2015).

A value having the data type Symbol can be referred to as a symbol value. Symbol is an immutable primitive value that is unique.

For example,

```
// two symbols with the same description
const value1 = Symbol('hello');
```

```
const value2 = Symbol('hello');
```

JavaScript Type

JavaScript is a dynamically typed (loosely typed) language. JavaScript automatically determines the variables' data type for supplied data. It also means that a variable can be of one data type and later it can be changed to another data type.

For example,

```
// data is of undefined type
let data;
// data is of integer type
data = 5;
```

JavaScript typeof

To find the type of a variable, the typeof operator is used.

For example,

```
const name = 'ram';
typeof(name); // returns "string"
```

Q. 10. Write a simple JavaScript program for login form validation. (6)

Ans:

```
<script>
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;
if (name==null || name==""){
  alert("Name can't be blank");
  return false;
}else if(password.length<6){
  alert("Password must be at least 6 characters long.");
  return false;
  }
}
</script>
<body>
<form name="myform" method="post" action="abc.jsp" onsubmit="return validateform()" >
Name: <input type="text" name="name"><br/>
Password: <input type="password" name="password"><br/>
<input type="submit" value="register">
</form>
```


OR

Q. 10. How to create a table in HTML. Explain with relevant - examples. (6)

Ans: HTML Tables : A **Table** in HTML is created using three basic tags: **table**, **tr**, and **td**. The tag **table** is the container of the whole table. The **opening tag <table>** tells the browser that a table is to be created. Table specific properties are also defined here. The **closing tag </table>** tells the browser that it is the end of the table. Everything else is contained within the **<table>** and **</table>** **tag pair**. So, a table always looks like this:

```
<table>
  ……
</table>
```

HTML table tags

Tag	Meaning
table	Represents the whole table
tr	Represents a row
td	Represents a cell in a row

th	Column header
caption	Title of the table

A table consists of a number of rows each of which is made up of a number of cells. The tag **<tr>** creates a **row**. The total number of rows in a table is equal to the number of **<tr>** elements. The opening tag **<tr>** and closing tag **</tr>** indicate the beginning and ending of a table row, respectively.

```
<table>
    <tr>….</tr>
    <tr>….</tr>
      …….
    <tr>….</tr>
</table>
```

A table or a row cannot contain data. To add data, cells are created within a row. Each cell in a row is defined by the tag **<td>**. The number of columns in a row is equal to the number of **<td>** elements. The content of a cell is written within the **<td>** and **</td>** tag pair. Textual as well as graphical content can be used in a cell. Usual tags such as **<p>**, **
, **<i>, **<b>**, and **<u>** may be used within a cell.

```
<table>
    <tr> <td>...</td> <td>...</td> <td>...</td> </tr>
    <tr> <td>...</td> <td>...</td> <td>...</td> </tr>
    ……
    <tr> <td>...</td> <td>...</td>…..<td>...</td> </tr>
</table>
```

Two other tags are defined: **<th>** and **<caption>**. To add a column header, the **<th>** tag is used. The title of a table is added using the **<caption>** tag.

The following code creates one such table:

```
<table>
    <tr> <td>82</td> <td>80</td> </tr>
    <tr> <td>80</td> <td>75</td> </tr>
</table>
```

Table Border

Adding a table border is very simple. The attribute **border** specifies that a border has to be used. Its value indicates the thickness of the outermost table boundary in **pixels**. A value **0 (zero)** indicates no border.

```
<table border="1">
    <tr> <td>82</td> <td>80</td> </tr>
    <tr> <td>80</td> <td>75</td> </tr>
</table>
```

Table Row and column header

Adding a title to a row/column helps to understand the meaning of data in a row/column. The tag **<th>** is used to add **row/column headers**. They are also added using **cells**. All column headers form a separate row that precedes all other rows. Similarly, all row headers form another column that precedes all other columns. Row/column headers are center-aligned and in bold font, which distinguishes them from other cells.

```
<table border="1">
    <tr> <th>Name</th> <th>MATH</th> <th>CPTC</th> <th>WT</th> </tr>
    <tr> <td>Sujay</td> <td>82</td> <td>80</td> <td>89</td> </tr>
    <tr> <td>Manish</td> <td>80</td> <td>75</td> <td>91</td> </tr>
</table>
```

Caption

It is a good idea to add a title/caption to a table that describes the table data. To add a caption to the table, the **<caption>** tag is used. For example, the following code creates a table with the caption "Marks".

```
<table border="1">
<caption>Marks</caption>
```

```
        <tr> <th>Name</th> <th>MATH</th> <th>CPTC</th> <th>WT</th> </tr>
        <tr> <td>Sujay</td> <td>82</td> <td>80</td> <td>89</td> </tr>
        <tr> <td>Manish</td> <td>80</td> <td>75</td> <td>91</td> </tr>
    </table>
```

The **<caption>** tag must appear only once and must appear immediately after the **<table>** tag and before the first **<tr>** tag.

The caption is centered with respect to the table itself. It may appear below or above the table. This can be specified by the **align** attribute. If nothing is mentioned, the caption is placed on the top of the table.

Rowspan and Colspan

To make a cell span multiple rows and columns, the tags **<rowspan>** and **<colspan>** are used, respectively. The tag **<rowspan>** indicates the number of rows a cell spans while **<colspan>** indicates the number of columns a cell spans.

```
<table border="1">
    <caption>Marks</caption>
    <tr>
    <th rowspan=2>Name</th>
    <th colspan=3>Marks</th>
    </tr>
    <tr> <th>MATH</th> <th>CPTC</th> <th>WT</th> </tr>
    <tr> <td>Sujay</td> <td>82</td> <td>80</td> <td>89</td> </tr>
    <tr> <td>Manish</td> <td>80</td> <td>75</td> <td>91</td> </tr>
</table>
```

Cellspacing and cellpadding

These attributes are used to adjust white spaces in your table. The attribute **cellspacing** specifies the amount of spaces, in pixels, between adjacent cells. Similarly, **cellpadding** specifies the amount of spaces in pixels between the cell border and its content.

```
<table border cellspacing="6" cellpadding="3" width=300>
<caption>Marks Table</caption>
<tr> <th rowspan=2>Name</th> <th colspan=3>Marks</th> </tr>
        <tr> <th>MATH</th> <th>CPTC</th> <th>WT</th> </tr>
        <tr> <td>Sujay</td> <td>82</td> <td>80</td> <td>89</td> </tr>
        <tr> <td>Manish</td> <td>80</td> <td>75</td> <td>91</td> </tr>
</table>
```


Q.11. Describe backgrounds and color gradient in CSS. (6)

Ans: Gradients are CSS elements of the image data type that show a transition between two or more colors. These transitions are shown as either linear or radial. Because they are of the image data type, gradients can be used anywhere an image might be. The most popular use for gradients would be in a background element.

CSS defines three types of gradients:

- **Linear Gradients (goes down/up/left/right/diagonally)**
- **Radial Gradients (defined by their center)**
- **Conic Gradients (rotated around a center point)**

CSS Linear Gradients

To create a linear gradient you must define at least two color stops. Color stops are the colors you want to render smooth transitions among. You can also set a starting point and a direction (or an angle) along with the gradient effect.

Syntax: **background-image: linear-gradient(direction, color-stop1, color-stop2, ...);**

Direction - Top to Bottom (this is default)

A linear gradient that starts at the top. It starts red, transitioning to yellow from top to bottom (default)

Example: #grad { background-image: linear-gradient(red, yellow); }

Direction - Left to Right

A linear gradient that starts from the left. It starts red, transitioning to yellow from left to right

Example: #grad { background-image: linear-gradient(to right, red , yellow); }

Direction - Diagonal

A gradient diagonally by specifying both the horizontal and vertical starting positions.

A linear gradient that starts at top left (and goes to bottom right). It starts red, transitioning to yellow from top left to bottom right

Example: #grad { background-image: linear-gradient(to bottom right, red, yellow); }

CSS Gradient Using Angles

More control over the direction of the gradient, it can define an angle, instead of the predefined directions (to bottom, to top, to right, to left, to bottom right, etc.). A value of 0deg is equivalent to "to top". A value of 90deg is equivalent to "to right". A value of 180deg is equivalent to "to bottom".

Syntax: **background-image: linear-gradient(angle, color-stop1, color-stop2);**

The following example shows how to use angles on linear gradients: 180deg

Example: #grad { background-image: linear-gradient(180deg, red, yellow); }

CSS Gradient Using Multiple Color Stops

The following example shows a linear gradient (from top to bottom) with multiple color stops:

Example: #grad { background-image: linear-gradient(red, yellow, green); }

The following example shows how to create a linear gradient (from left to right) with the color of the rainbow and some text: Rainbow Background

Example: #grad { background-image: linear-gradient(to right, red,orange,yellow,green,blue,indigo,violet); }

CSS Gradient Using Transparency

CSS gradients also support transparency, which can be used to create fading effects.

To add transparency, we use the rgba() function to define the color stops. The last parameter in the rgba() function can be a value from 0 to 1, and it defines the transparency of the color: 0 indicates full transparency, 1 indicates full color (no transparency).

The following example shows a linear gradient that starts from the left. It starts fully transparent, transitioning to full color red:

Example: #grad { background-image: linear-gradient(to right, rgba(255,0,0,0), rgba(255,0,0,1)); }

CSS Gradient Repeating a linear-gradient

The repeating-linear-gradient() function is used to repeat linear gradients:

Example

A repeating linear gradient:

Example: #grad { background-image: repeating-linear-gradient(red, yellow 10%, green 20%); }

OR

Q.11. Briefly explain the control flow statements in JavaScript with examples. (6)

Ans: The control flow is the order in which the computer executes statements in a script. Code is run in order from the first line in the program to the last line, unless the computer runs across the structures that change the control flow, such as conditionals and loops. JavaScript has two types of control statements - Conditional and Iterative (Looping) with their particular uses. All of the logic and flow control is achieved using some very simple structures. These are:

- Conditional Statements
  - if statements
  - if ... else ... statements
- Looping Statements
  - while loops
  - do ... while loops
- switch Statements
- label Statements
- with Statements

JavaScript Conditions Statements

While writing a program, there may be a situation when need to adopt one out of a given set of paths. In such cases, use conditional statements that allow program to make correct decisions and perform right actions. JavaScript supports conditional statements which are used to perform different actions based on different conditions.

JavaScript supports the following forms of **conditional** statements −

- **if statement**
- **if...else statement**
- **if...else if... statement.**

if statement

The **if** statement is the fundamental control statement that allows JavaScript to make decisions and execute statements conditionally.

Syntax

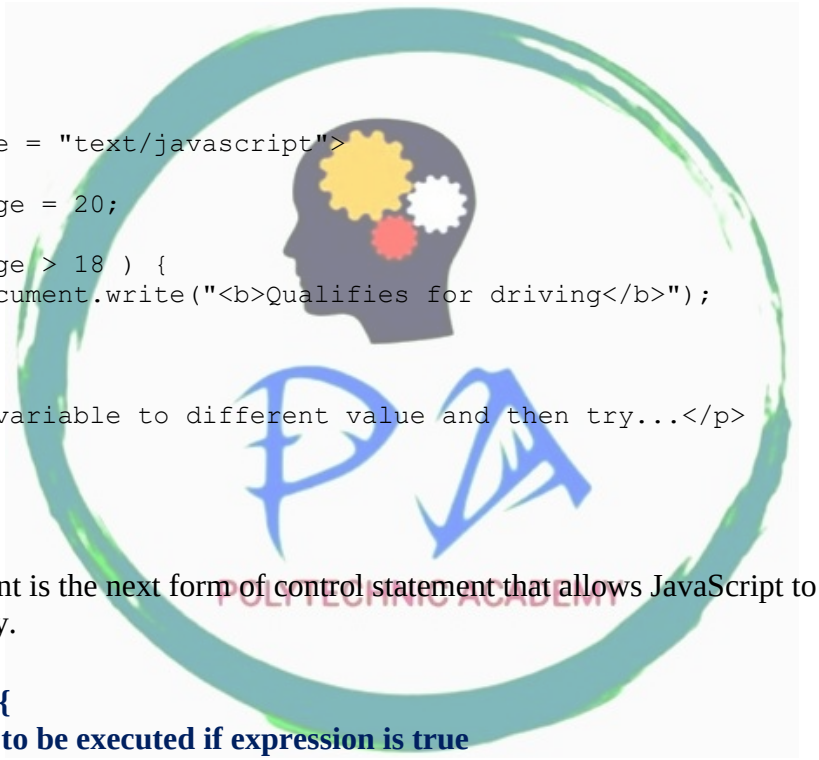
The syntax for a basic if statement is as follows −

```
if (expression) {
  Statement(s) to be executed if expression is true
}
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var age = 20;

            if( age > 18 ) {
               document.write("<b>Qualifies for driving</b>");
            }
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```



if...else statement

The '**if...else**' statement is the next form of control statement that allows JavaScript to execute statements in a more controlled way.

Syntax

```
if (expression) {
  Statement(s) to be executed if expression is true
} else {
  Statement(s) to be executed if expression is false
}
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var age = 15;
            if( age > 18 ) {
               document.write("<b>Qualifies for driving</b>");
            } else {
               document.write("<b>Does not qualify for driving</b>");
            }
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
```

```
</html>
```

if...else if... statement

The **if...else if...** statement is an advanced form of **if…else** that allows JavaScript to make a correct decision out of several conditions.

Syntax

The syntax of an if-else-if statement is as follows −

```
if (expression 1) {
  Statement(s) to be executed if expression 1 is true
} else if (expression 2) {
  Statement(s) to be executed if expression 2 is true
} else if (expression 3) {
  Statement(s) to be executed if expression 3 is true
} else {
  Statement(s) to be executed if no expression is true
}
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var book = "maths";
            if( book == "history" ) {
               document.write("<b>History Book</b>");
            } else if( book == "maths" ) {
               document.write("<b>Maths Book</b>");
            } else if( book == "economics" ) {
               document.write("<b>Economics Book</b>");
            } else {
               document.write("<b>Unknown Book</b>");
            }
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
<html>
```


JavaScript – Loop Structures

While writing a program, in a situation where need to perform an action over and over again. In such situations, need to write loop statements to reduce the number of lines.

JavaScript supports all the necessary loops to ease down the pressure of programming.

The while Loop

The purpose of a **while** loop is to execute a statement or code block repeatedly as long as an **expression** is true. Once the expression becomes **false,** the loop terminates.

Syntax

The syntax of **while loop** in JavaScript is as follows −

```
while (expression) {
  Statement(s) to be executed if expression is true
}
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var count = 0;
            document.write("Starting Loop ");
```

```
            while (count < 10) {
               document.write("Current Count : " + count + "<br />");
               count++;
            }
            document.write("Loop stopped!");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```

The do...while Loop

The **do...while** loop is similar to the **while** loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is **false**.

Syntax

The syntax for **do-while** loop in JavaScript is as follows −

```
do {
   Statement(s) to be executed;
} while (expression);
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var count = 0;
            document.write("Starting Loop" + "<br />");
            do {
               document.write("Current Count : " + count + "<br />");
               count++;
            }
            while (count < 5);
            document.write ("Loop stopped!");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```


JavaScript - For Loop

The '**for**' loop is the most compact form of looping. It includes the following three important parts −

- The **loop initialization** where counter is initialized to a starting value. The initialization statement is executed before the loop begins.
- The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.
- The **iteration statement** where counter increases or decreases.

All the three parts are put in a single line separated by semicolons.

Syntax

The syntax of **for** loop is JavaScript is as follows −

```
for (initialization; test condition; iteration statement) {
   Statement(s) to be executed if test condition is true
}
```

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var count;
            document.write("Starting Loop" + "<br />");
```

```
            for(count = 0; count < 10; count++) {
               document.write("Current Count : " + count );
               document.write("<br />");
            }
            document.write("Loop stopped!");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```

JavaScript for...in loop

The **for...in** loop is used to loop through an object's properties.

Syntax

The syntax of 'for..in' loop is −

```
for (variablename in object) {
  statement or block to execute
}
```

In each iteration, one property from **object** is assigned to **variablename** and this loop continues till all the properties of the object are exhausted.

Example

Try the following example to implement 'for-in' loop. It prints the web browser's **Navigator** object.

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var aProperty;
            document.write("Navigator Object Properties<br /> ");
            for (aProperty in navigator) {
               document.write(aProperty);
               document.write("<br />");
            }
            document.write ("Exiting from the loop!");
         //-->
      </script>
      <p>Set the variable to different object and then try...</p>
   </body>
</html>
```


JavaScript - Switch Case

The objective of a **switch** statement is to give an expression to evaluate and several different statements to execute based on the value of the expression. The interpreter checks each **case** against the value of the expression until a match is found. If nothing matches, a **default** condition will be used.

Syntax

```
switch (expression) {
    case condition 1: statement(s)
    break;
    case condition 2: statement(s)
    break;
    ...
    case condition n: statement(s)
    break;
    default: statement(s)
}
```

The **break** statements indicate the end of a particular case. If they were omitted, the interpreter would continue executing each statement in each of the following cases.

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var grade = 'A';
            document.write("Entering switch block<br />");
            switch (grade) {
               case 'A': document.write("Good job<br />");
               break;
               case 'B': document.write("Pretty good<br />");
               break;
               case 'C': document.write("Passed<br />");
               break;
               case 'D': document.write("Not so good<br />");
               break;
               case 'F': document.write("Failed<br />");
               break;
               default:  document.write("Unknown grade<br />")
            }
            document.write("Exiting switch block");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```

JavaScript - Loop Control

JavaScript provides full control to handle loops and switch statements. There may be a situation when need to come out of a loop without reaching its bottom. There may also be a situation when want to skip a part of code block and start the next iteration of the loop.

To handle all such situations, JavaScript provides **break** and **continue** statements. These statements are used to immediately come out of any loop or to start the next iteration of any loop respectively.

The break Statement

The **break** statement, which was briefly introduced with the *switch* statement, is used to exit a loop early, breaking out of the enclosing curly braces.

Syntax: break;

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
         var x = 1;
         document.write("Entering the loop<br /> ");
         while (x < 20) {
            if (x == 5) {
               break;   // breaks out of loop completely
            }
            x = x + 1;
            document.write( x + "<br />");
         }
         document.write("Exiting the loop!<br /> ");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```

The continue Statement

The **continue** statement tells the interpreter to immediately start the next iteration of the loop and skip the remaining code block. When a **continue** statement is encountered, the program flow moves to the loop

check expression immediately and if the condition remains true, then it starts the next iteration, otherwise the control comes out of the loop.

Example

This example illustrates the use of a **continue** statement with a while loop. Notice how the **continue** statement is used to skip printing when the index held in variable **x** reaches 5 −

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            var x = 1;
            document.write("Entering the loop<br /> ");

            while (x < 10) {
               x = x + 1;
               if (x == 5) {
                  continue;    // skip rest of the loop body
               }
               document.write( x + "<br />");
            }
            document.write("Exiting the loop!<br /> ");
         //-->
      </script>
      <p>Set the variable to different value and then try...</p>
   </body>
</html>
```

Using Labels to Control the Flow

A **label** is simply an identifier followed by a colon (:) that is applied to a statement or a block of code.
Note − Line breaks are not allowed between the **'continue'** or **'break'** statement and its label name. Also, there should not be any other statement in between a label name and associated loop.

Example

```
<html>
   <body>
      <script type = "text/javascript">
         <!--
            document.write("Entering the loop!<br /> ");
            outerloop:        // This is the label name
            for (var i = 0; i < 5; i++) {
               document.write("Outerloop: " + i + "<br />");
               innerloop:
               for (var j = 0; j < 5; j++) {
                  if (j > 3 ) break ;           // Quit the innermost loop
                  if (i == 2) break innerloop;  // Do the same thing
                  if (i == 4) break outerloop;  // Quit the outer loop
                  document.write("Innerloop: " + j + " <br />");
               }
            }
            document.write("Exiting the loop!<br /> ");
         //-->
      </script>
   </body>
</html>
```

****